koroteevmv / ML_course
Code Issues 2 Pull requests Agents Actions Projects Security and quality Insights
Type / to search

Files
main
Go to file
ML0.1 intro
ML0.2 simple analisys
ML1.1 linear regression
ML1.2 multivariate
ML1.3 real regression
ML2.1 logistic regression
ML2.2 real classification
ML3.1 polynomial features
ML3.2 svm
ML3.3 mlp
ML3.4 trees
ML3.5 knn
ML3.6 bayes
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration

ML_course / ML3.4 trees /
Add file
koroteev-test Картинки распределены по папкам 7dd1f73 · 2 years ago History

Name	Last commit message	Last commit date
..		
img	Картинки распределены по папкам	2 years ago
README.md	Картинки распределены по папкам	2 years ago

README.md

Деревья решений

Цель работы

Научиться применять модель дерева принятия решений для задач классификации и регрессии.

Содержание работы

1. Сгенерируйте данные для задачи классификации на три класса при помощи sklearn.datasets.make_blobs, содержащие два признака и 100 наблюдений. Визуализируйте их на диаграмме рассеяния.
2. Обучите модель классификационного решающего дерева sklearn.svm.DecisionTreeClassifier глубины 4 и критерием entropy.
3. Визуализируйте соответствующий граф дерева решений.
4. Визуализируйте полученные разделяющие области.
5. Оцените качество работы модели. Создайте новое наблюдение и сделайте предсказание на нём.
6. Сгенерируйте данные для задачи регрессии, содержащие один признак и 100 наблюдений, как случайный шум некоторой функции. Визуализируйте их на диаграмме рассеяния.
7. Обучите модель регрессионного решающего дерева sklearn.svm.DecisionTreeRegressor глубиной 3.
8. Визуализируйте соответствующий граф дерева решений.
9. Визуализируйте получившуюся кусочную линию регрессии.
10. Оцените качество работы модели. Создайте новое наблюдение и сделайте предсказание на нём.

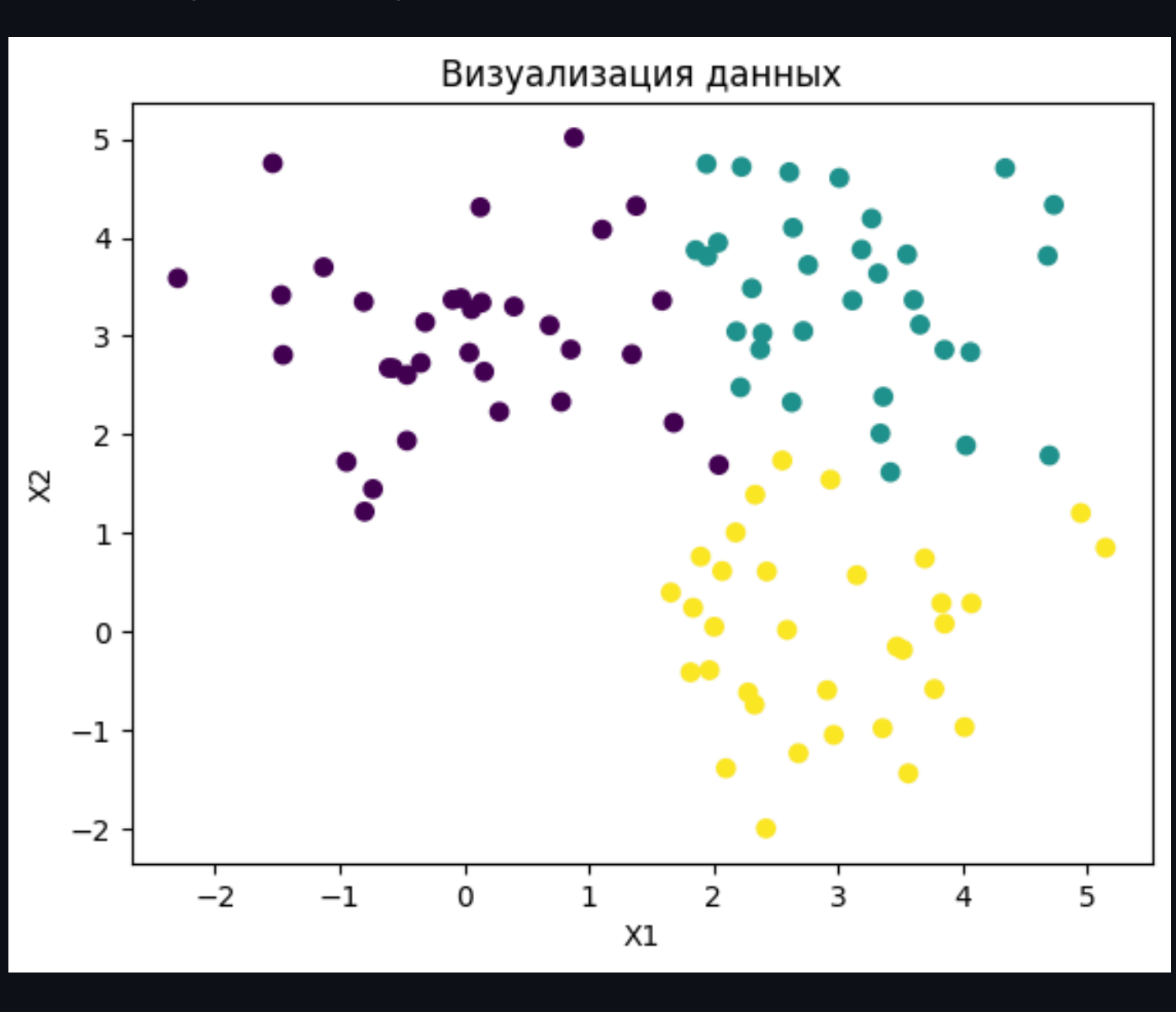
Методические указания

Задача классификации

По аналогии с предыдущими работами, создадим и визуализируем датасет:

```
X, y = make_blobs(n_samples=100, centers=[(0,3),(3,3),(3,0)],
                                        n_features=2, random_state=RANDOM_SEED,
                                        cluster_std=(0.9,0.9,0.9))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title('Визуализация данных')
plt.xlabel('X1')
plt.ylabel('X2')
```

Видим набор данных из трех классов:



Также, как и перцептрон, деревья решений могут легко применяться к задачам разной размерности. Количество классов не играет роли для дерева. Содадим и обучим модель:

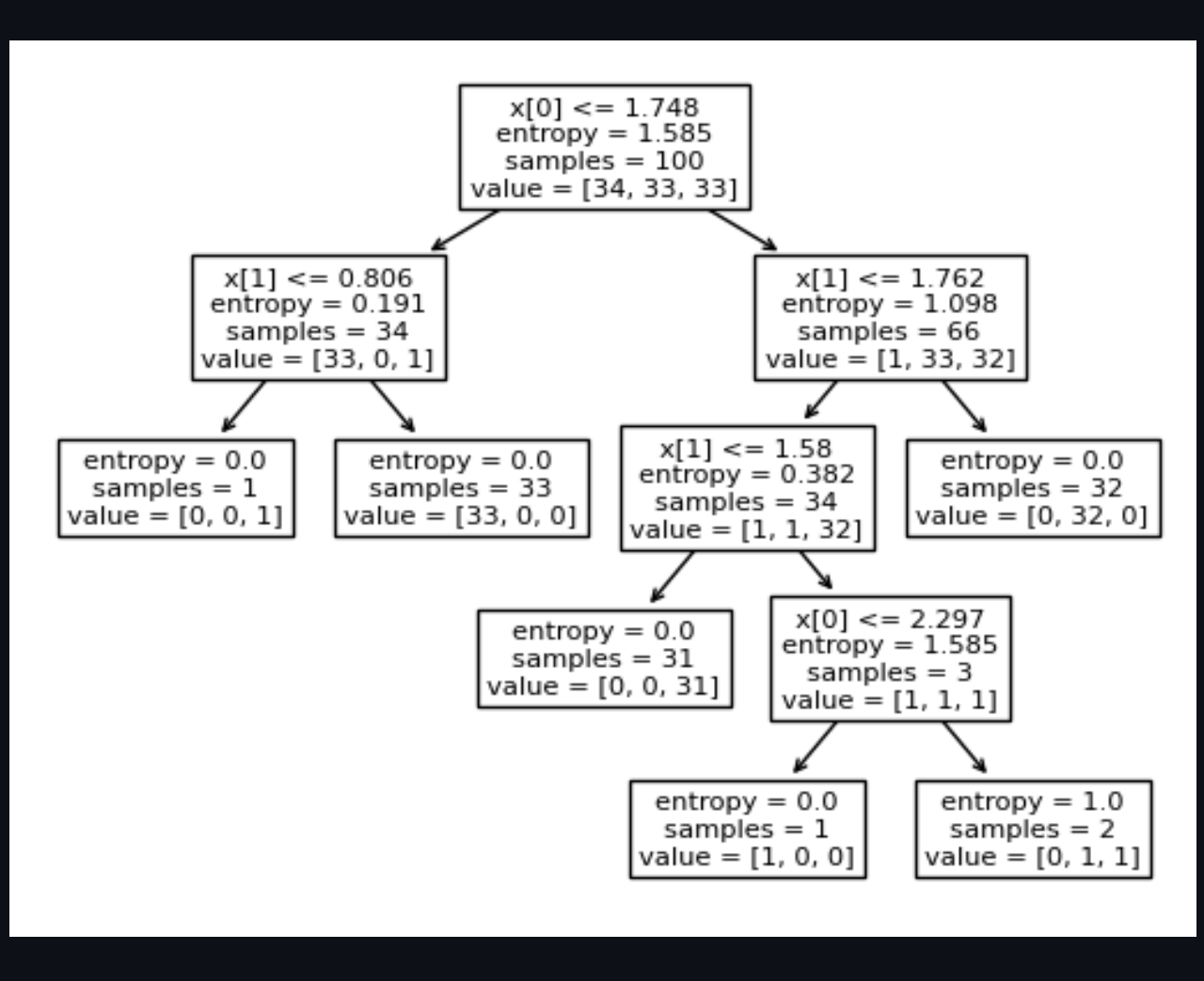
```
depth=4
clf_tree = DecisionTreeClassifier(criterion='entropy', max_depth=depth,
                                  random_state=RANDOM_SEED)
clf_tree.fit(X, y)
```

Здесь мы задаем вид функции, которая будет использоваться для нахождения оптимальной границы разбиения выборки. По умолчанию используется критерий Джини, а мы сейчас будем использовать критерий информационной энтропии. Также мы задает максимальную глубину дерева. Дерево не будет "расти" дальше этого количества уровней. Вы самостоятельно можете изменить значения этих параметров и проанализировать, как это повлияет на рабоут модели.

После обучения данной модели мы можем вывести собственно само дерево решений, то есть полное внутреннее устройство модели. Для этого в объекте существует специальный метод *plot_tree()*:

```
tree.plot_tree(clf_tree)
plt.show()
```

В этом объекте есть еще один метод визуализации дерева - втекстовом виде. Самостоятельно найдите и примените его. А использованный нами метод выводит дерево в графическом виде:


x[0] <= 1.748
entropy = 1.585
samples = 100
value = [34, 33, 33]
x[1] <= 0.806
entropy = 0.191
samples = 34
value = [33, 0, 1]
x[1] <= 1.762
entropy = 1.098
samples = 66
value = [1, 33, 32]
entropy = 0.0
samples = 1
value = [0, 0, 1]
entropy = 0.0
samples = 33
value = [33, 0, 0]
x[1] <= 1.58
entropy = 0.382
samples = 34
value = [1, 1, 32]
entropy = 0.0
samples = 32
value = [0, 32, 0]
entropy = 0.0
samples = 31
value = [0, 0, 31]
x[0] <= 2.297
entropy = 1.585
samples = 3
value = [1, 1, 1]
entropy = 0.0
samples = 1
value = [1, 0, 0]
entropy = 1.0
samples = 2
value = [0, 1, 1]

Проинтерпретируйте изображенную на этом графике информацию.

Теперь можно визуализировать сами границы принятия решений:

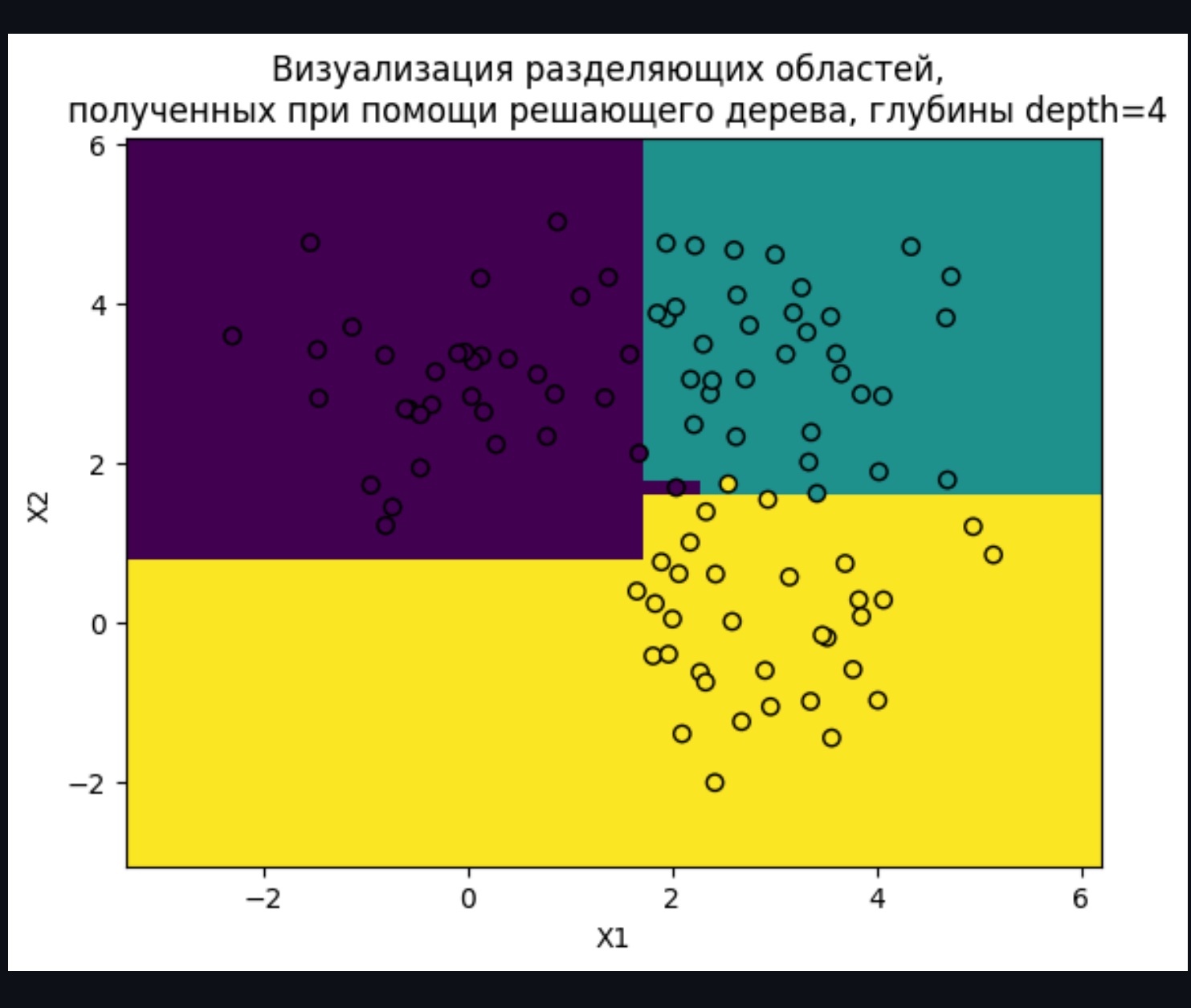
```
X0 = np.linspace(X[:, 0].min()-1,X[:, 0].max()+1, X.shape[0])
X1 = np.linspace(X[:, 1].min()-1,X[:, 1].max()+1, X.shape[0])
X0_grid, X1_grid = np.meshgrid(X0, X1)

y_predict = clf_tree.predict(np.c_[X0_grid.ravel(),X1_grid.ravel()]).reshape(X0_grid.shape)
plt.pcolormesh(X0_grid, X1_grid, y_predict)

plt.scatter(X[:, 0], X[:, 1], c=y,  edgecolors='black',linewidth=1)

plt.title('Визуализация разделяющих областей, \n полученных при помощи решающего дерева, глубины depth={}'.format(depth))
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
```

Мы видим очень характерную для деревьев форму границы:



Как всегда оценим качество работы модели и с помощью метрик:

```
y_pred = clf_tree.predict(X)
print(confusion_matrix(y, y_pred))
print('Accuracy =', accuracy_score(y, y_pred))
print('F1_score =', f1_score(y, y_pred, average='micro'))
```

```
[[34  0  0]
 [ 0 33  0]
 [ 0  1 32]]
Accuracy = 0.99
F1_score = 0.99
```

Мы видим, что модель допустила всего одну ошибку на 100 примерах, что довольно неплохо. Но увеличив глубину дерева можно еще повысить его эффективность и добиться абсолютной точности.

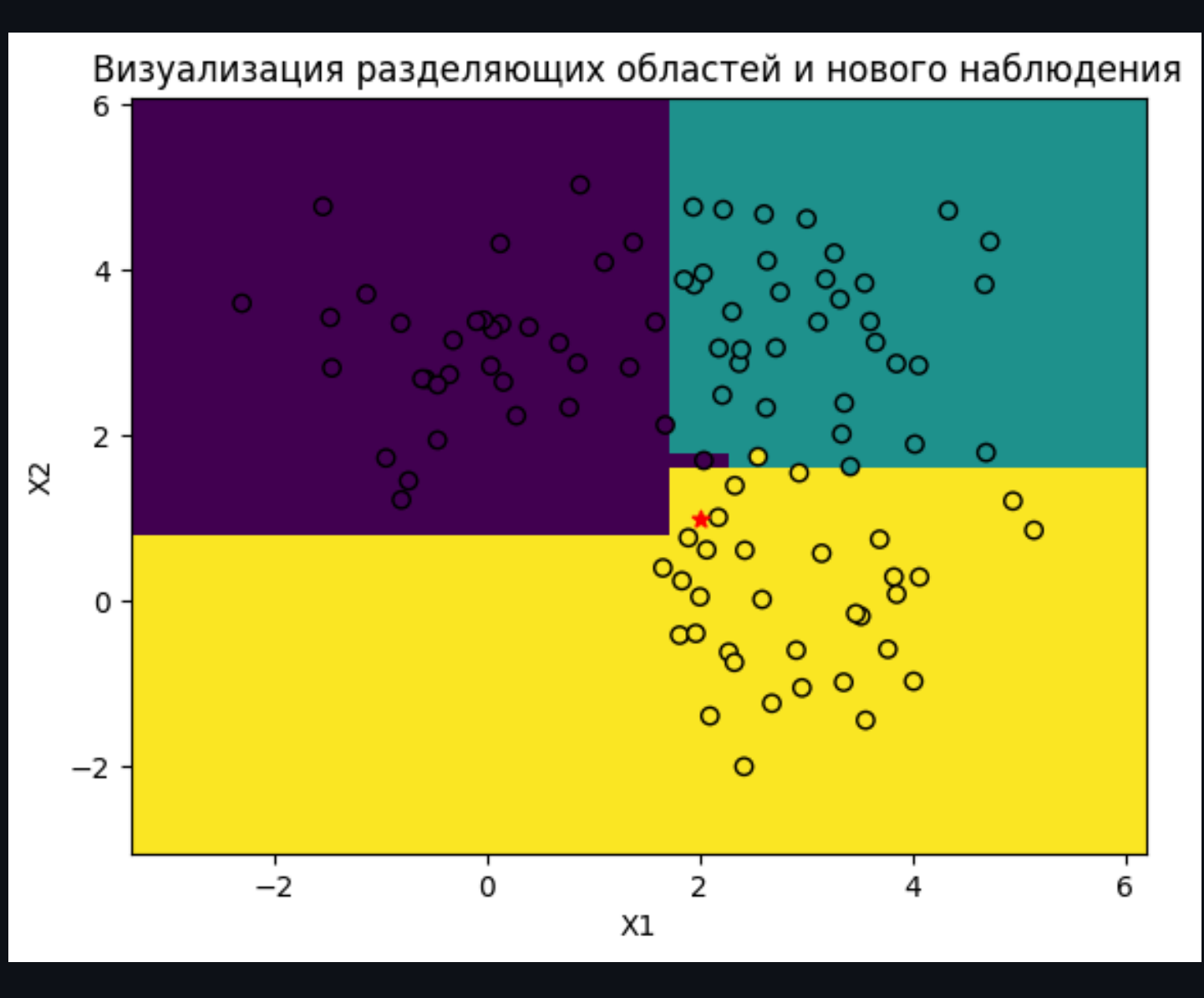
Теперь создадим новое наблюдение:

```
observation_new = [[2, 1]]
```

И классифицируем его:

```
clf_tree.predict(observation_new)
```

Визуализируем на графике его вместе с границей принятия решений:

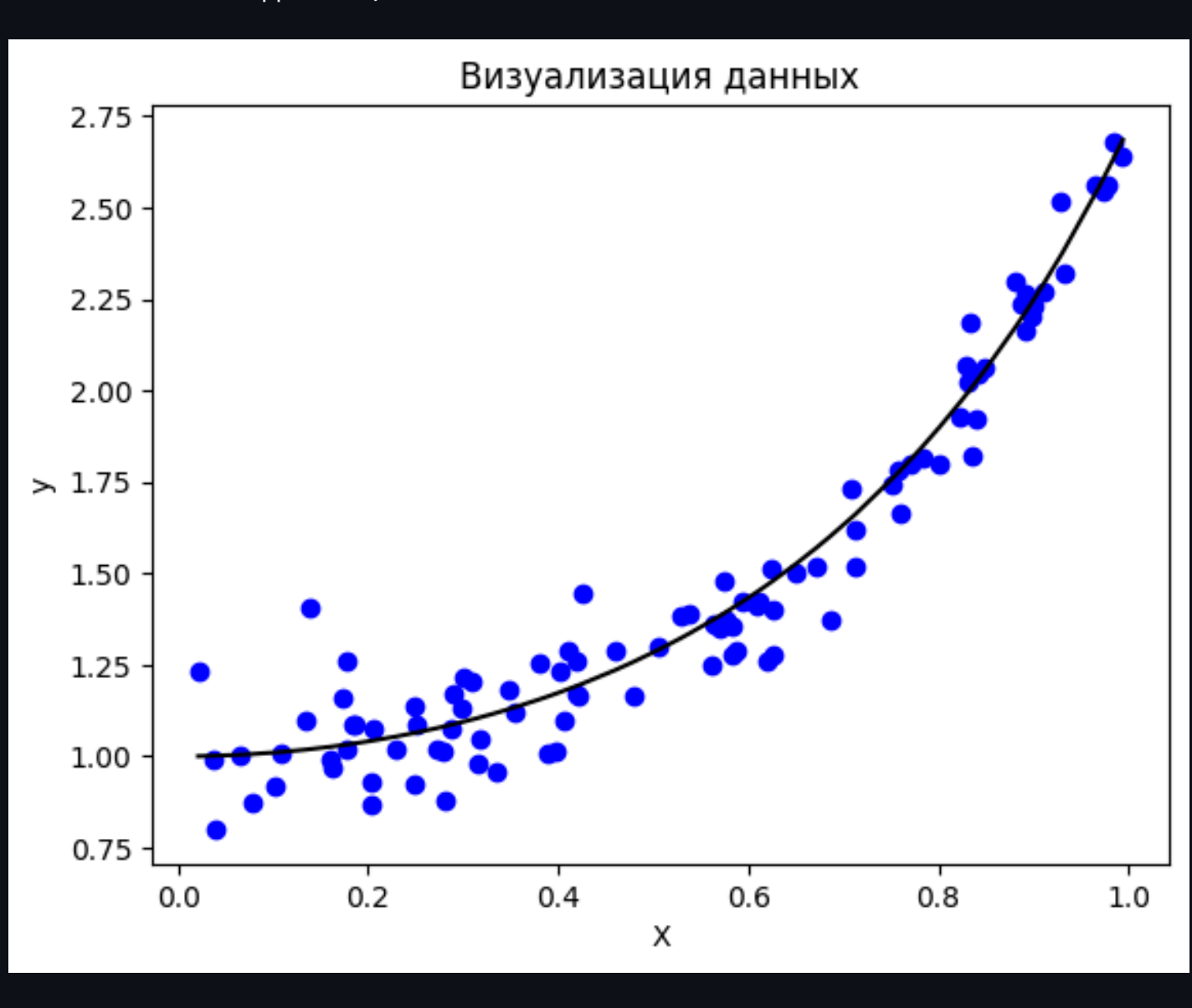


Задача регрессии

Сгенерируем простой датасет для задачи регрессии. Для этого получим 100 упорядоченных случайных чисел, а затем значения целевой переменной вычислим как результат какой-нибудь функции (например возьмем экспоненту) и прибавим к результату случайный шум, для имитации разброса:

```
n_samples = 100
X = np.sort(np.random.rand(n_samples))
y = np.exp(X ** 2)+np.random.normal(0.0, 0.1, X.shape[0])
```

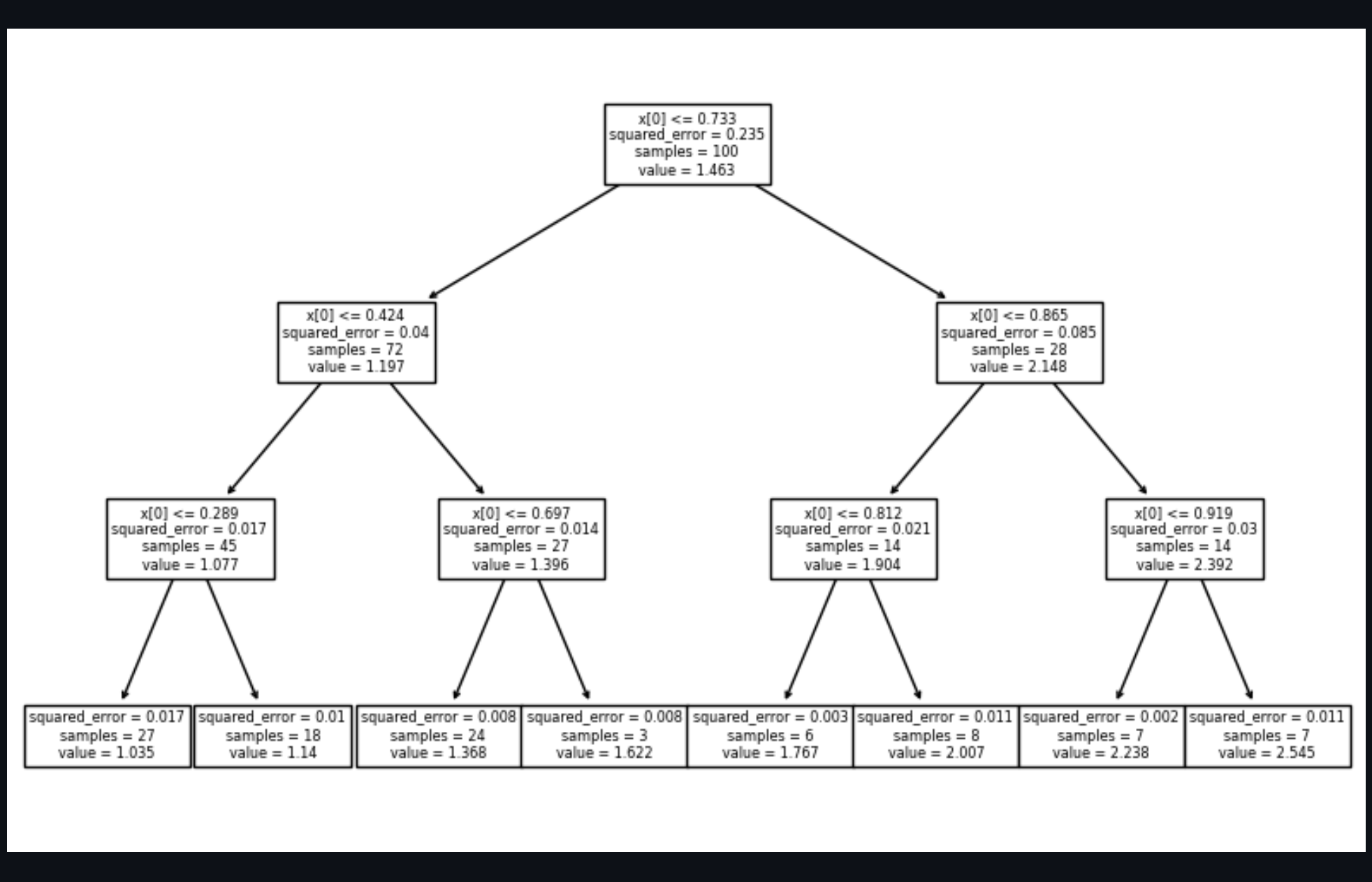
Мы получим примерно такой датасет для парной регрессии (линия здесь добавлена только для информации и наглядности, она не является частью данных):



Создадим объект регрессора на основе деревьев решений и обучим его:

```
depth=3
reg_tree = DecisionTreeRegressor(max_depth=depth, random_state=RANDOM_SEED)
reg_tree.fit(X, y)
```

Выведем полученное дерево:


x[0] <= 0.733
squared_error = 0.235
samples = 100
value = 1.463
x[0] <= 0.424
squared_error = 0.04
samples = 72
value = 1.197
x[0] <= 0.865
squared_error = 0.085
samples = 28
value = 2.148
x[0] <= 0.289
squared_error = 0.017
samples = 45
value = 1.077
x[0] <= 0.697
squared_error = 0.014
samples = 27
value = 1.396
x[0] <= 0.812
squared_error = 0.021
samples = 14
value = 1.904
x[0] <= 0.919
squared_error = 0.03
samples = 14
value = 2.392
squared_error = 0.017
samples = 27
value = 1.035
squared_error = 0.01
samples = 18
value = 1.14
squared_error = 0.008
samples = 24
value = 1.368
squared_error = 0.008
samples = 3
value = 1.622
squared_error = 0.003
samples = 6
value = 1.767
squared_error = 0.011
samples = 8
value = 2.007
squared_error = 0.002
samples = 7
value = 2.238
squared_error = 0.011
samples = 7
value = 2.545

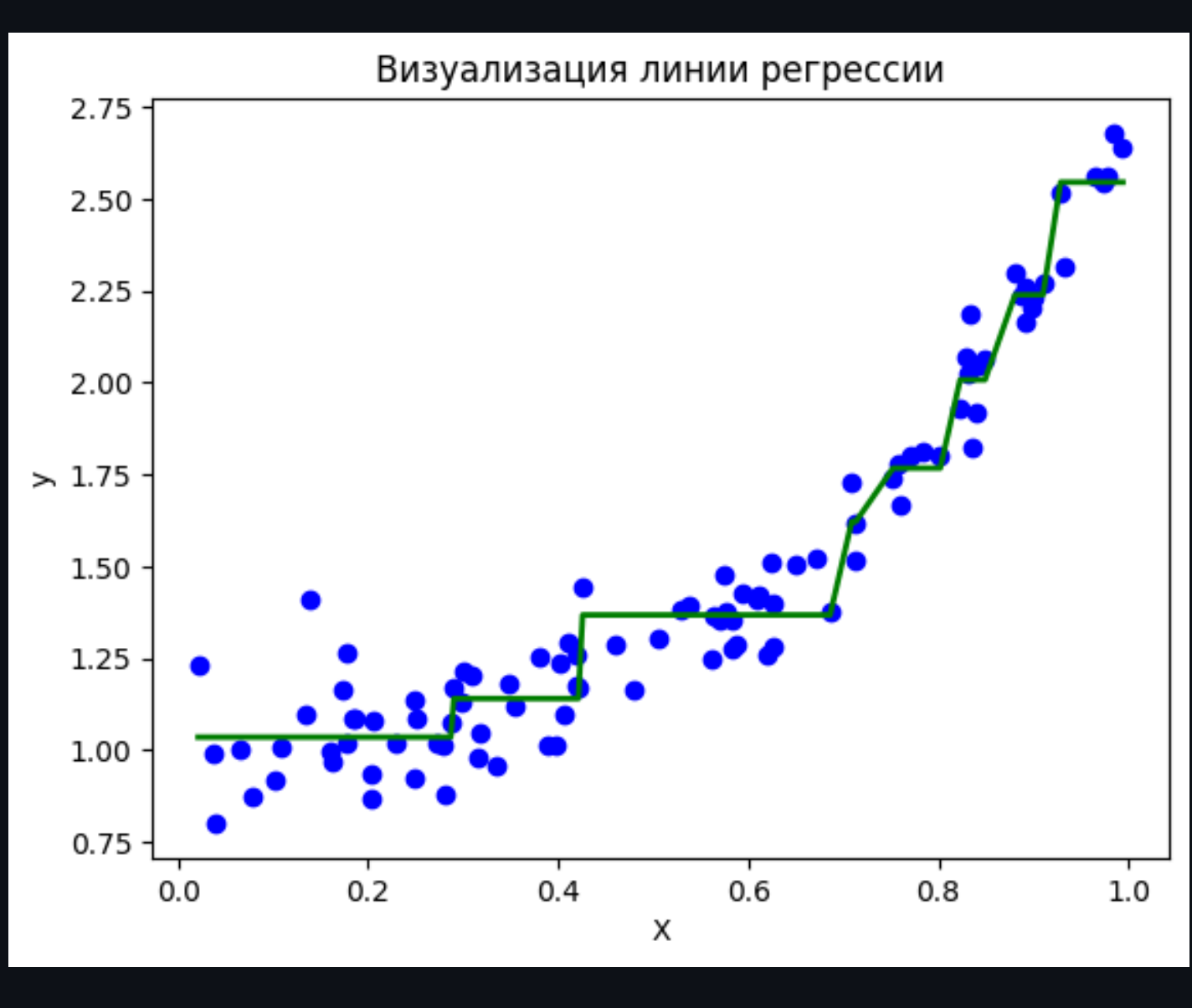
Самостоятельно проинтерпретируйте информацию в этом графе. А мы построим линию регрессии на графике::

```
plt.scatter(X, y, c="b")

plt.plot(X, y_pred_reg, "g", lw=2)

plt.title('Визуализация линии регрессии')
plt.xlabel('X')
plt.ylabel('y');
```

Опять видим характерную кусочно-линейную линию:



Тперь как всегда оценим качество работы модели:

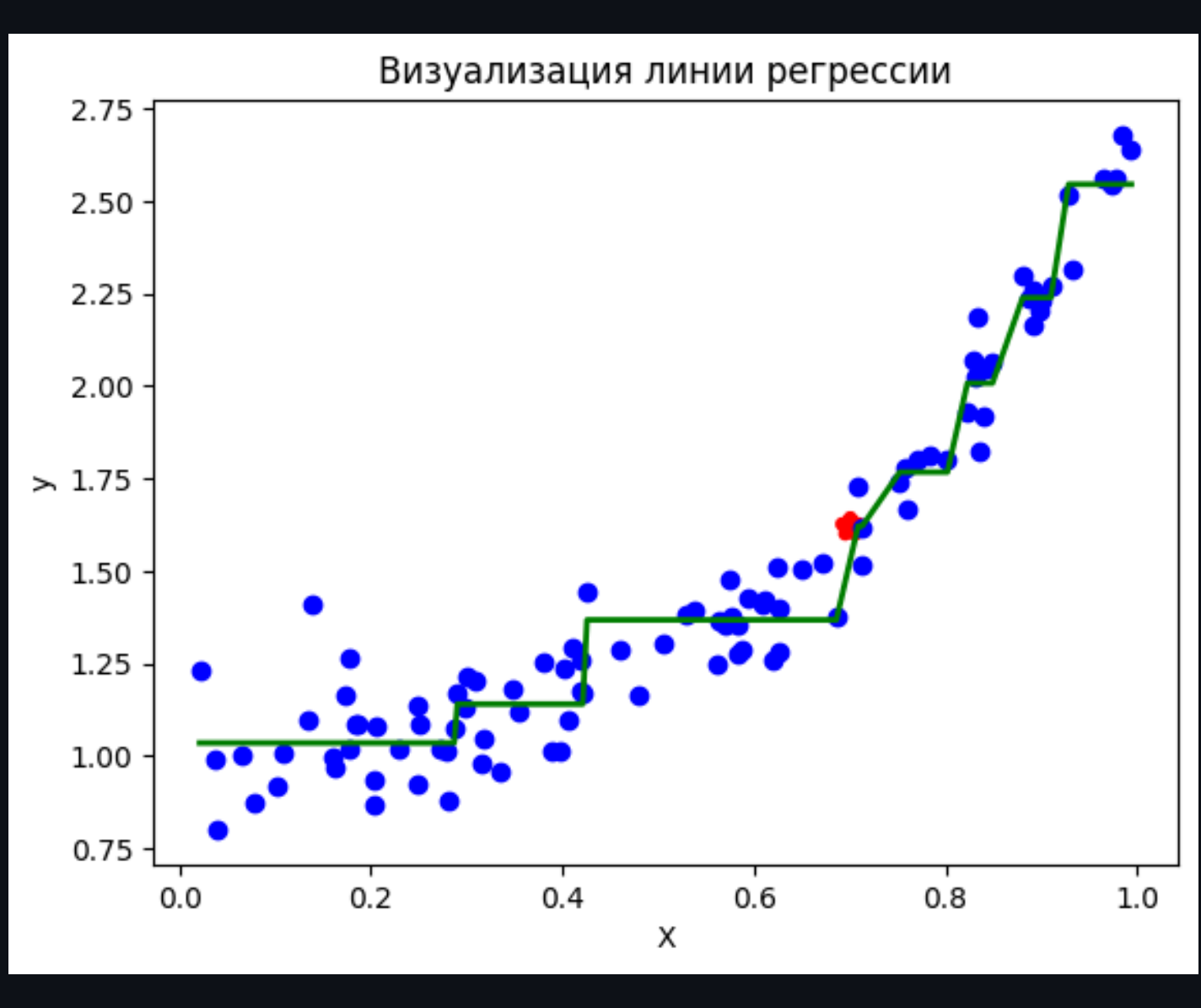
```
print('r2= ', r2_score(y, y_pred_reg))
print('MSE= ', mean_squared_error(y, y_pred_reg))
```

```
r2=  0.9559340585697235
MSE=  0.010347452773751169
```

Данные метрики свидетельствуют о довольно точной работе модели. Лишь на графике открывается определенная условность и неглабкость полученных предсказаний. Давайте создадим новое предсказание:

```
observation_new = [[0.7]]
```

И изобразим его на графике:



Самостоятельно подвигайте данную точку. Обратите внимание, что есть участки, где небольшое изменение положения точки не изменяет результат предсказания.

Задания для самостоятельного выполнения

1. Загрузите встроенные данные sklearn.datasets.load_iris, взяв только последние два столбца (длина и ширина лепестков). Изобразите их на диаграмме рассеяния, подкрасив каждый класс некоторым цветом.
2. Обучите модель классификационного дерева принятия решений sklearn.tree.DecisionTreeClassifier глубины 4, используя энтропию. Визуализируйте соответствующий граф дерева принятия решений.
3. Обучите модель классификационного дерева принятия решений sklearn.tree.DecisionTreeClassifier с разными глубинами (1, 2, 3, 4, 10), используя энтропию, и визуализируйте в каждом случае полученные разделяющие области.
4. Выведите необходимые метрики для оценки работы моделей с разными глубинами. Сделайте вывод о том, какая модель лучше классифицирует данные.
5. Загрузите весь датасет load_iris. Обучите модель классификационного дерева принятия решений sklearn.tree.DecisionTreeClassifier глубины 4, используя энтропию. Визуализируйте соответствующий граф дерева решений. Оцените качество работы модели.
6. Загрузите встроенные данные sklearn.datasets.california_housing, взяв только столбец AveBedrms в качестве единственного признака. Изобразите данные на диаграмме рассеяния так, чтобы на одной оси были отмечены значения признака, а на другой - целевой переменной.
7. Обучите модель регрессионного дерева принятия решений sklearn.tree.DecisionTreeRegressor, зафиксировав random_state=0, а остальными гиперпараметрами по умолчению.
8. Визуализируйте соответствующий граф дерева решений и получившуюся кусочную линию регрессии.
9. Оцените качество работы модели. Создайте новое наблюдение и сделайте предсказание на нём.
10. Загрузите весь датасет fetch_california_housing. Обучите ту же модель. Визуализируйте соответствующий граф дерева решений и оцените качество работы модели.

Контрольные вопросы

1. Почему граница принятия решений у деревьев решений имеет такую характерную форму?
2. Как глубина дерева влияет на сложность модели?
3. Почему глубина дерева на разных ветках может быть разная?
4. Что такое критерий в деревьях решений и как он влияет на работу модели?

Дополнительные задания

1. Проверьте работу модели на ненормализованных данных с очень разной величиной. Сделайте вывод о применимости модели дерева решений без нормализации
2. Примените дерево решений на большом датасете (не менее 10 000 строк и 10 столбцов) по вашему выбору.
3. Повторите моделирование с другим критерием. Найдите датасеты, на которых лучше работают разные критерии.
4. Используйте на первом датасете вместо деревьев решений случайный лес. Визуализируйте его границу принятия решений.